IMEN574 / IDSC574 · PROGRAMMING FOR AX

17

Visual Representations

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Visual Representations

01 **Encoding Images**

From pixels to tokens

02 **Learning Representations**

From pairs to features

03 **Transferring Features**

From features to tasks

01 · Encoding Images

01 **Encoding Images**
From pixels to tokens

02 **Learning Representations**
From pairs to features

03 **Transferring Features**
From features to tasks

Why Representations

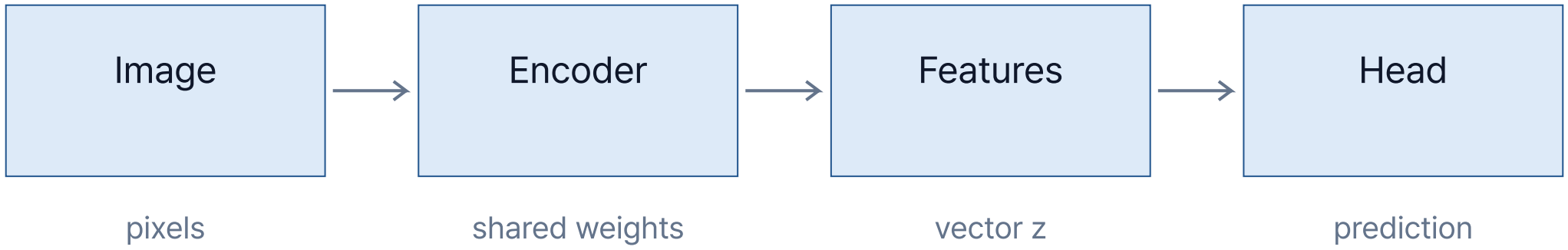
Pixel distance changes when the object barely changes

Image A	0	0	1	0
Shifted A	0	1	0	0
Background	0	0	0	0

A one-pixel shift changes the array

Representation and Readout

The encoder supplies features to a task-specific head



A new head can reuse the same features

Global and Spatial Features

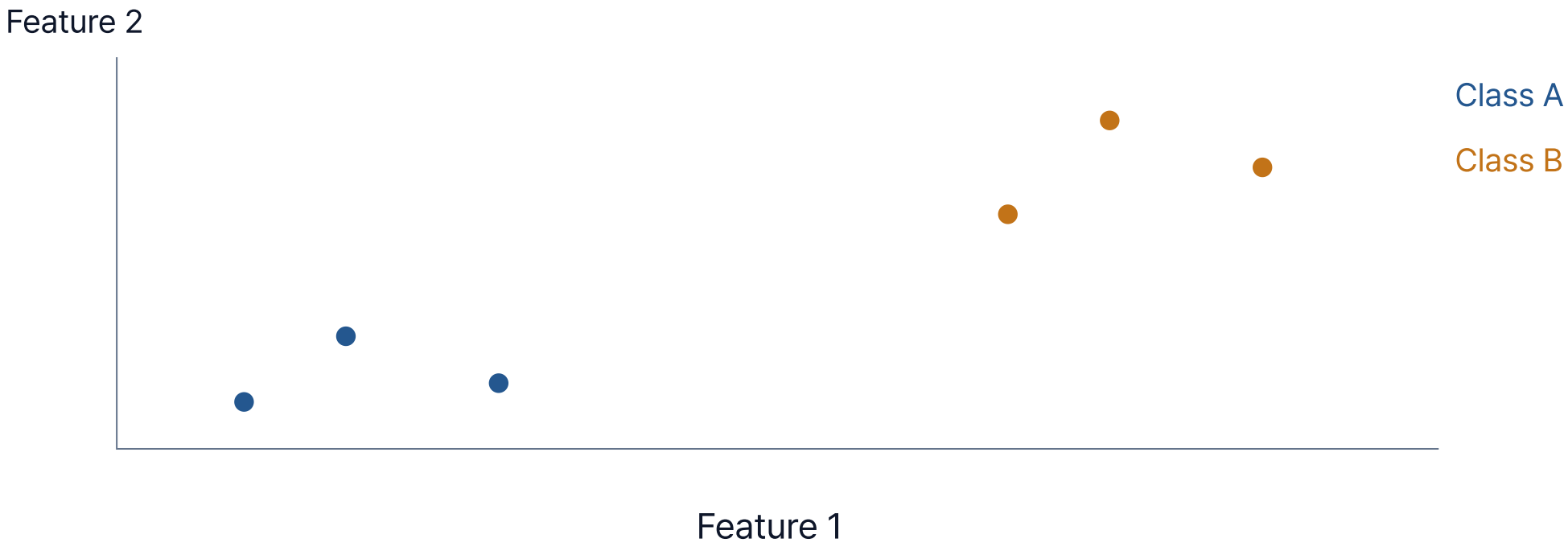
Pooling trades location for a compact summary

Patch 1	1	0	0	1
Patch 2	0	1	1	0

Pooling removes the identity of each patch

Supervised Visual Learning

Labels shape which differences matter



Nearby embeddings share the training label

Local Receptive Fields

A convolution applies one local rule across an image

top	0	1	0
middle	1	4	1
bottom	0	1	0

The same kernel visits each local neighborhood

Convolution Output Size

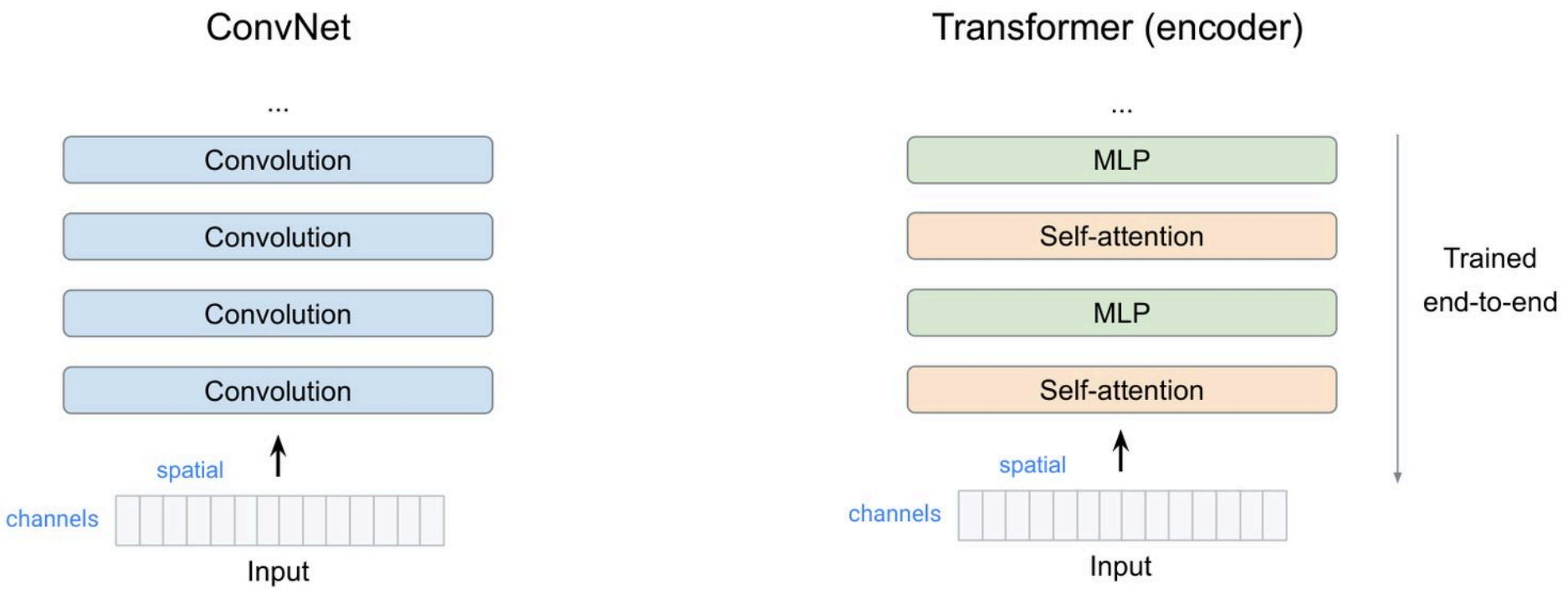
Stride and padding determine the spatial grid

```
H_out = floor((H + 2P − K) / S) + 1
```



Convolution and Attention

Local filters and global token mixing impose different structures



Token mixing changes which locations can interact

https://arxiv.org/abs/2010.11929

Patch Embeddings

An image becomes a sequence of patch vectors

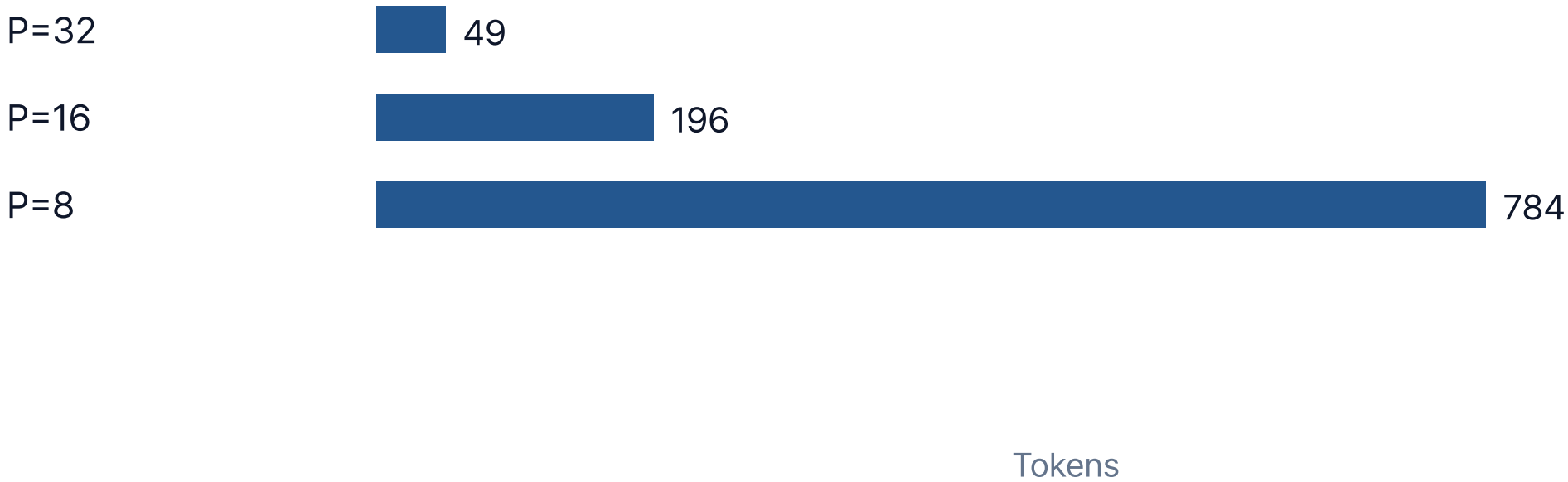
Image patches	p1	p2	p3	p4
Embeddings	z1	z2	z3	z4

Each patch becomes one token

Patch Count

Smaller patches create longer sequences

$$N = (H / P) \times (W / P)$$



Worked Example: Patch Tokens

Trace the inputs before execution

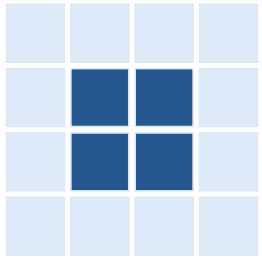
```
4×4 image; center 2×2 pixels = 1
Patch width = 2; horizontal shift = 0
Flatten each nonoverlapping patch in row-major order.
```

Demo: Patch Tokens

Executable educational calculation; no pretrained inference

Change patch width from 2 to 4; compare token count and token width

Patch width 2 Horizontal shift 0



4 tokens × 4 values

Token 0: [0, 0, 0, 1]

Token 1: [0, 0, 1, 0]

Token 2: [0, 1, 0, 0]

Token 3: [1, 0, 0, 0]

Patch means

0.250 0.250 0.250 0.250

4×4 image → 4 tokens; token width=4.

Questions: Does shifting preserve tokens? Does mean pooling preserve position?

Answer: Patch Tokens

Same inputs as the initial demo state

```
4 tokens, each with 4 values
each patch mean = .25
```

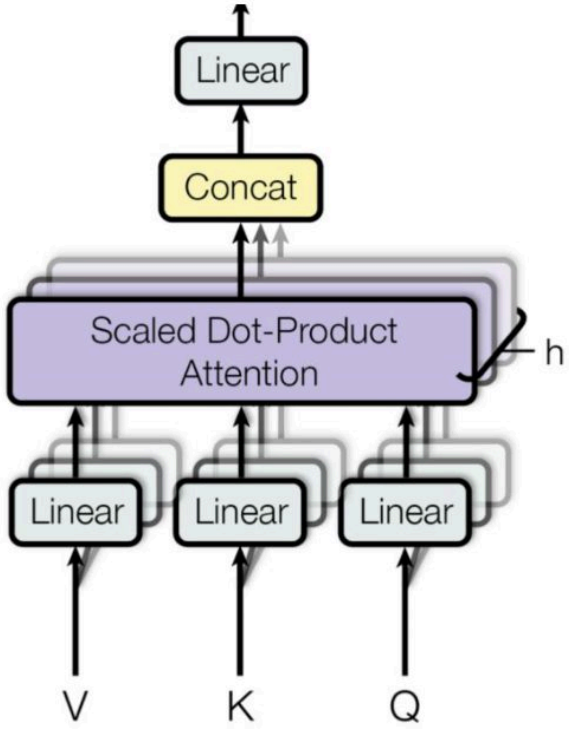
Failure: Patch Tokens

Identify the assumption that breaks

```
Mean pooling discards within-patch arrangement
patch tokens are not learned semantic features
```

Vision Transformer

Patch tokens enter a shared Transformer encoder



Patch tokens share the same encoder block

https://arxiv.org/abs/1706.03762

Worked Example: CLS Attention Mask

Trace the inputs before execution

```
4×4 patch logits: central four=2, other twelve=0
Apply softmax across all 16 patches
Select weights ≥ .05; target mask is central four.
```

Demo: CLS Attention Mask

Executable educational calculation; no pretrained inference

Raise the threshold from 0.05 to 0.20; compare mask size and target IoU

Attention threshold 0.05

CLS query scores → softmax across 16 patches

0.024	0.024	0.024	0.024
0.024	0.178	0.178	0.024
0.024	0.178	0.178	0.024
0.024	0.024	0.024	0.024

Selected patches=4; target IoU=1.000

Target = central four patches

Thresholded attention is a hypothesis, not ground truth

Fixed educational CLS logits: center=2, others=0; all weights computed by softmax.
Questions: Does a threshold change the learned features? Can an empty mask be a valid foreground?

Answer: CLS Attention Mask

Same inputs as the initial demo state

```
Central weight .177809, background .024064
selected4 patches
IoU1
```

Failure: CLS Attention Mask

Identify the assumption that breaks

```
Attention thresholding changes a proposed mask, not representation learning
this synthetic example does not establish SAM-quality segmentation
```

Demo: Attention Thresholds

Educational 4×4 attention logits replace pretrained DINO attention

Run | Code | Data

Original model in Colab ↗

Educational calculation · Original: DINO attention maps

Input JSON

```
{
  "scores": [0, 0, 0, 0, 0, 2, 2,
0, 0, 2, 2, 0, 0, 0, 0, 0],
  "temperature": 1,
  "threshold": 0.1
}
```

Run here | Stop | Reset

0.024	0.024	0.024	0.024
0.024	0.178	0.178	0.024
0.024	0.178	0.178	0.024
0.024	0.024	0.024	0.024

```
Attention weights
[0.0241, 0.0241, 0.0241, 0.0241]
[0.0241, 0.1778, 0.1778, 0.0241]
[0.0241, 0.1778, 0.1778, 0.0241]
[0.0241, 0.0241, 0.0241, 0.0241]

Weight sum: 1.0000
Selected patches: 4 / 16
```

Recorded browser run · Run to recompute

Change temperature and threshold separately. Which changes the attention weights?

Position Information

Permutation alone cannot recover the original layout

	Left	Right
Top	top-left	top-right
Bottom	bottom-left	bottom-right

Position identifies where each patch came from

02 · Learning Representations

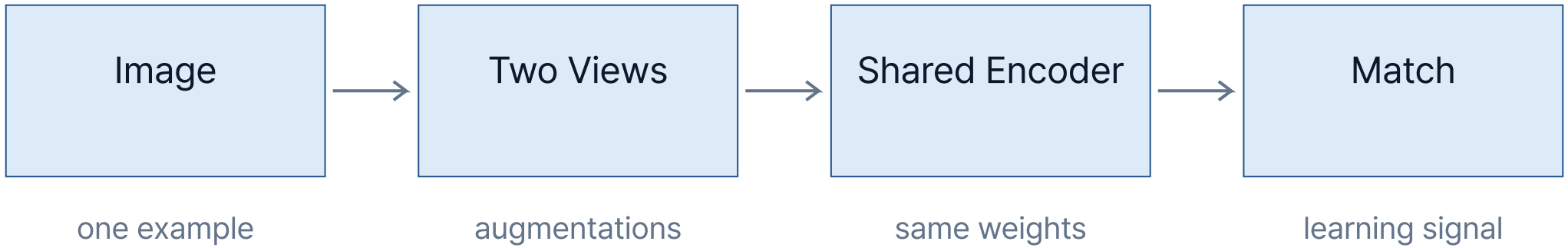
01 **Encoding Images**
From pixels to tokens

02 **Learning Representations**
From pairs to features

03 **Transferring Features**
From features to tasks

Why Learn Without Labels

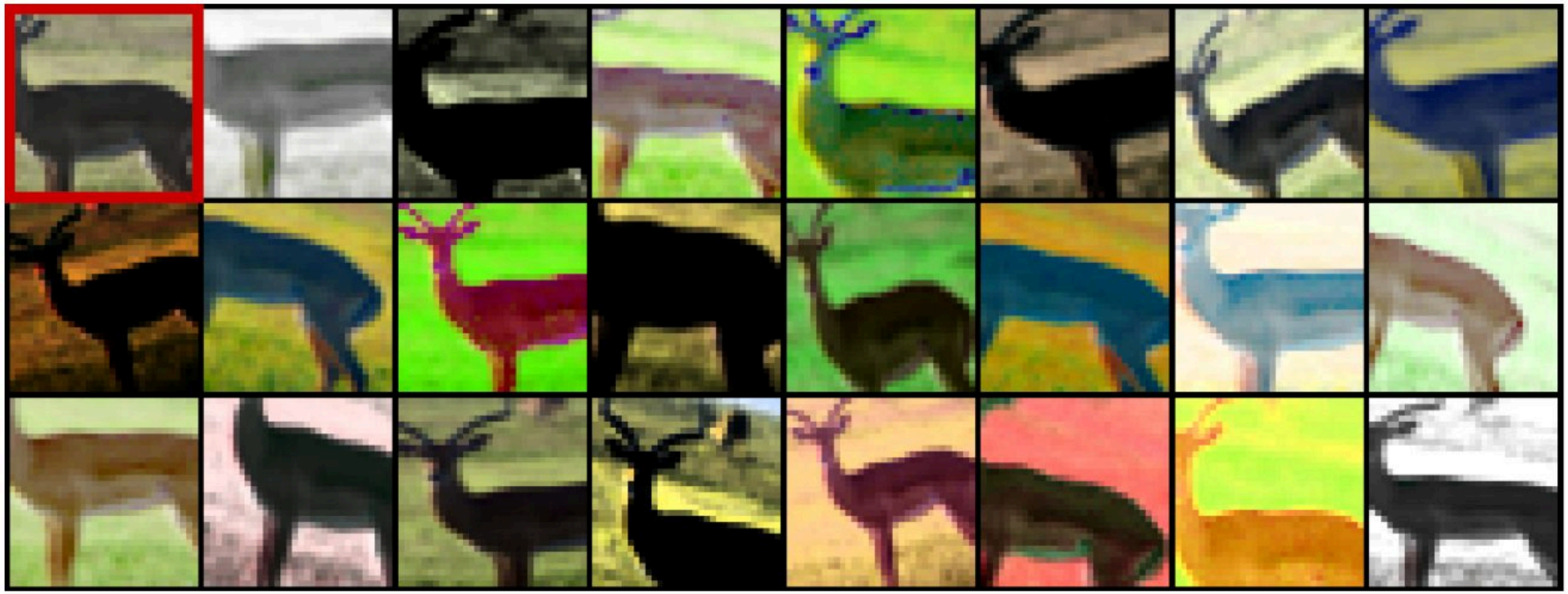
Unlabeled images still provide paired observations



Two views provide a learning relation

Visual Cues

Different crops can preserve the same object



Different views can retain one object identity

Positive and Negative Pairs

A training relation defines which features move together



Pair labels define the contrastive task

Cosine Similarity

Normalization separates direction from vector magnitude

$$\cos(z, v) = z^{T}v \,/\, (||z||\ ||v||)$$

A	1	0
2A	2	0
B	0	1

Similarity Matrix

Aligned views should score above mismatched pairs

	view 1	view 2	view 3
image 1	0.95	0.1	0.2
image 2	0.15	0.9	0.05
image 3	0.1	0.25	0.92

The diagonal marks matched observations

Contrastive Objective

The matching example competes with alternatives

$$L_i = -\log[\exp(s_{ii} / \tau) / \Sigma_j \exp(s_{ij} / \tau)]$$

Representation Collapse

Identical vectors satisfy matching without preserving information

Cat	1	0
Car	1	0
Tree	1	0

Perfect agreement can erase every difference

Avoiding Collapse

Additional constraints keep representations informative

Family	Constraint
Contrastive	Compare other examples
Distillation	Teacher targets and asymmetry
Redundancy reduction	Control covariance

Worked Example: Paired-View Loss

Trace the inputs before execution

```
Views: [1,0], [0,1], [1,1] in both branches
L2-normalize; compare all 3×3 pairs; temperature = 1
Loss = −mean log softmax(similarity)[paired index]
```

Demo: Paired-View Loss

Executable educational calculation; no pretrained inference

Compare Distinct with Collapsed features; explain why collapse has nonzero loss

Temperature 1 Features Distinct

Paired-image similarity

1.000	0.000	0.707
0.000	1.000	0.707
0.707	0.707	1.000

Mean contrastive loss = 0.803

Positive P = 0.473, 0.473, 0.401

Diagonal entries identify paired views

Unit-vector dot products → row softmax → −mean log P(pair).
Questions: Can collapse give zero loss? What does temperature change?

Answer: Paired-View Loss

Same inputs as the initial demo state

```
Mean contrastive loss = .803438
collapsed loss = log(3) = 1.098612
```

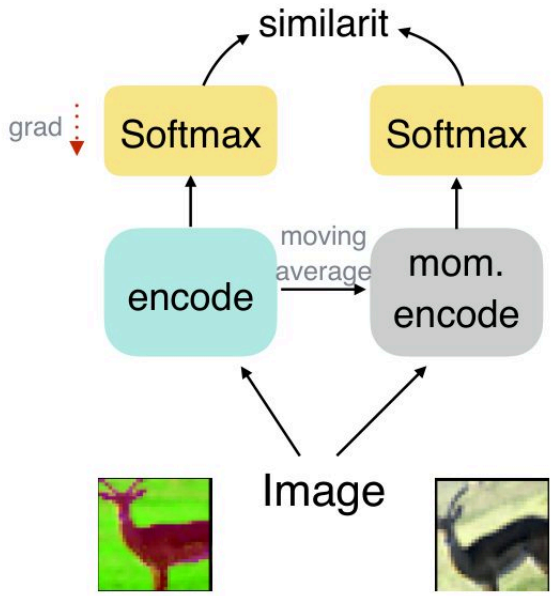
Failure: Paired-View Loss

Identify the assumption that breaks

```
A low small-batch loss does not establish transfer quality
duplicated positives can become false negatives
```

Teacher and Student

A teacher supplies targets for another augmented view



The target comes from another view of the image

03 · Transferring Features

01 Encoding Images

From pixels to tokens

02 Learning Representations

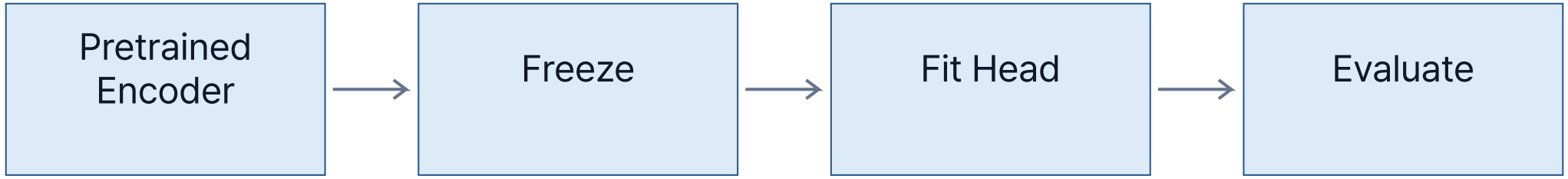
From pairs to features

03 **Transferring Features**

From features to tasks

Frozen Feature Transfer

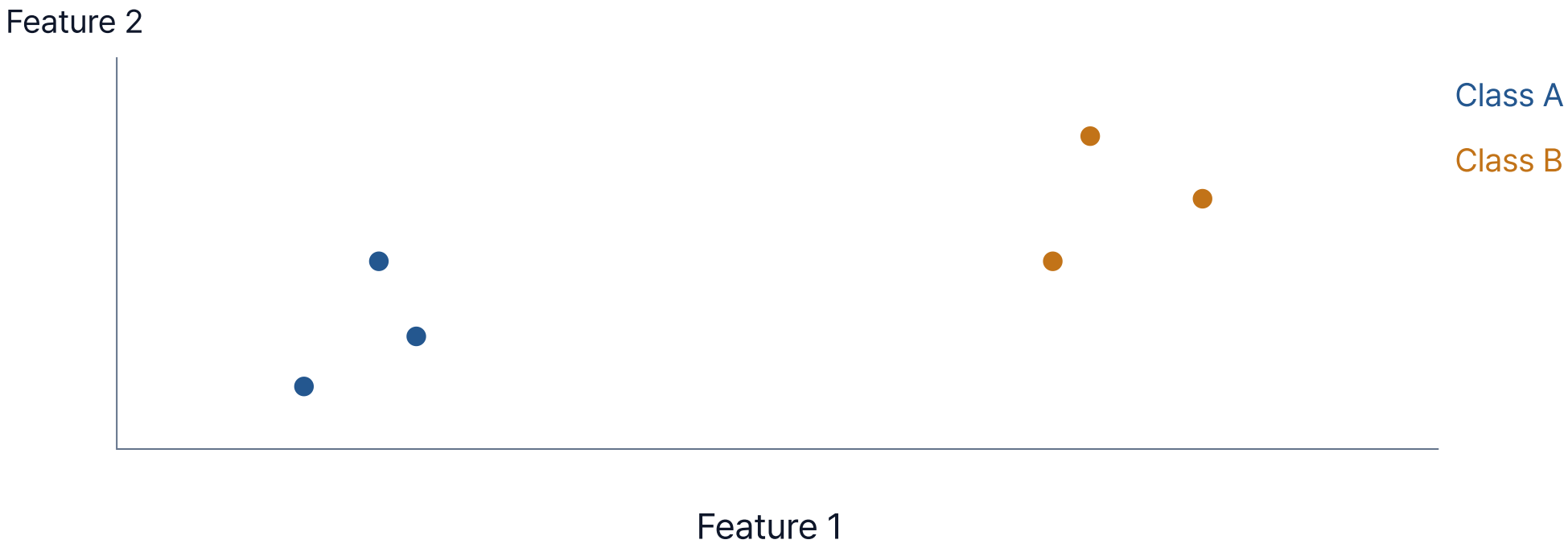
A small readout can reuse a pretrained encoder



Only the readout changes in a frozen probe

Linear Probing

A linear head tests what is already accessible



A simple boundary exposes separable information

Worked Example: Frozen-Feature Probe

Trace the inputs before execution

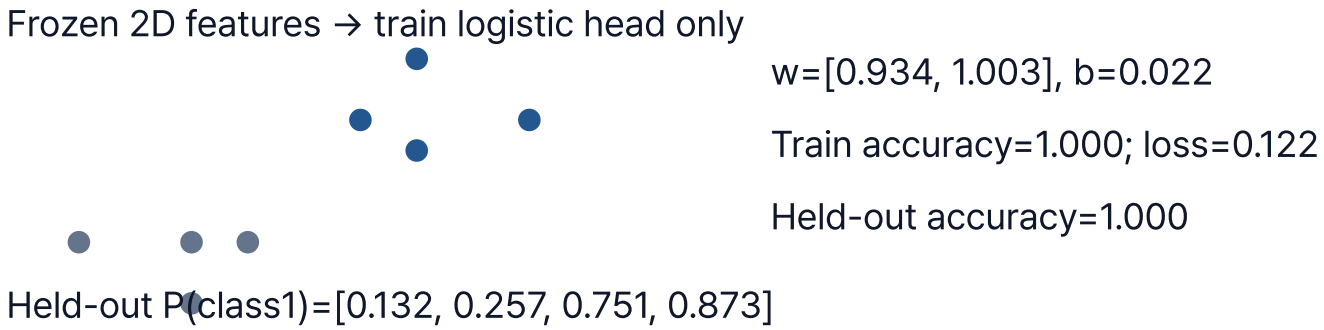
```
Fixed train features: 4 negative points, 4 positive points
Train logistic weights only; initialize w=[0,0], b=0
20 full-batch steps, learning rate .2; hold out 4 separate points.
```

Demo: Frozen-Feature Probe

Executable educational calculation; no pretrained inference

Train the head for 20 steps; shift held-out x by 3 and compare both accuracies

Gradient steps 20 Learning rate 0.2 Held-out x shift 0



Full-batch gradient descent on 8 train rows; 4 held-out rows; no DINO checkpoint.
Questions: Which weights change? Why can held-out feature shift break a good probe?

Answer: Frozen-Feature Probe

Same inputs as the initial demo state

```
w=[.933529,1.002926], b=.021973
training loss .121807
train accuracy1 and held-out accuracy1
```

Demo: DINOv2 Linear Probe

DINOv2-small replaces DINO ViT-S/16; 6 train images and 2 held-out images

Run | Code | Data

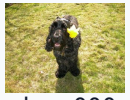
Original model in Colab ↗

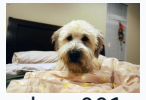
Pretrained model · weights ≈ 24 MB · Original: DINO ViT-S/16

Input JSON

```
{
  "images": ["dog_000",
"dog_001", "dog_002", "cat_000",
"cat_001", "cat_002", "dog_003",
"cat_003"],
  "labels": [0, 0, 0, 1, 1, 1, 0,
1],
  "steps": 100,
  "rate": 0.5
}
```

Run here | Stop | Reset

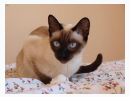

dog_000


dog_001


dog_002


cat_000


cat_001


cat_002


dog_003


cat_003

```
Training rows: 6
Held-out rows: 2
Held-out accuracy: 1.0000

Held-out P(cat)
Row 6: 0.2557; label dog
Row 7: 0.5977; label cat

Only the linear head was trained
```

Recorded browser run · Run to recompute

Change gradient steps. Which weights change, and which rows stay held out?

Failure: Frozen-Feature Probe

Identify the assumption that breaks

```
Frozen features do not mean the classifier is pretrained
toy probe accuracy does not measure DINO performance
```

Fine-Tuning Scope

Updating more parameters increases adaptation and validation needs



Illustrative trainable parameter units

More trainable weights expand the adaptation space

Augmentation Choice

Preserving the wrong invariant can remove the target

Color	red	green
Meaning	stop	go

Removing color can remove the label

Caveat: Background Shortcut

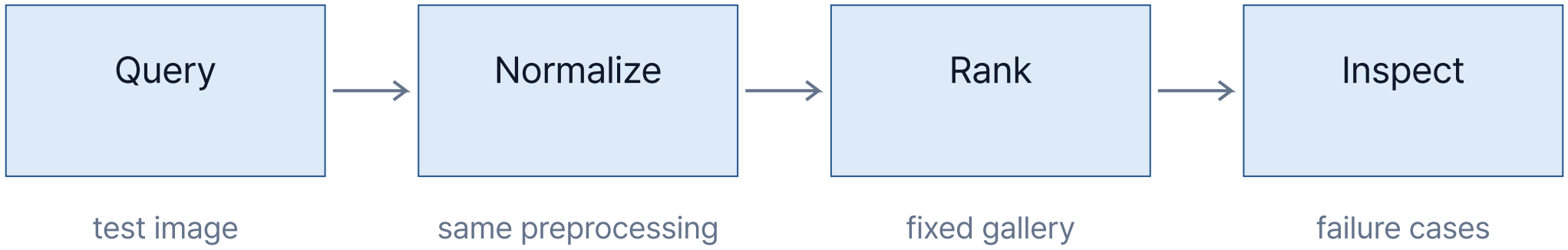
A classifier may learn context instead of the object

	Class A	Class B
Train: snow	8	0
Train: grass	0	8
Test: mixed	4	4

A background shortcut fails when context changes

Retrieval Inspection

Nearest examples reveal useful and spurious similarities



The gallery determines what retrieval can find

Pooling Code

Spatial tokens and global vectors have different shapes

```
tokens = np.arange(24).reshape(2, 3, 4)
global_vectors = tokens.mean(axis=1)
assert global_vectors.shape == (2, 4)
```

Feature Normalization

Zero-length embeddings need an explicit policy

```
norm = np.linalg.norm(z, axis=-1, keepdims=True)
z_unit = z / np.maximum(norm, 1e-12)
```

Demo: DINOv2 Feature Geometry

DINOv2-small replaces DINO; compute cosine directly, without t-SNE

Run | Code | Data

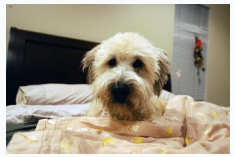
Original model in Colab ↗

Pretrained model · weights ≈ 24 MB · Original: DINO ViT-S/16

Input JSON

```
{
  "images": ["dog_000",
"dog_001", "cat_000", "cat_001"]
}
```


dog_000


dog_001


cat_000


cat_001

```
Cosine matrix
dog_000 | dog_001 | cat_000 | cat_001

[1.0000, 0.0797, 0.0275, 0.0377]
[0.0797, 1.0000, -0.0205, -0.0641]
[0.0275, -0.0205, 1.0000, 0.0239]
[0.0377, -0.0641, 0.0239, 1.0000]
```

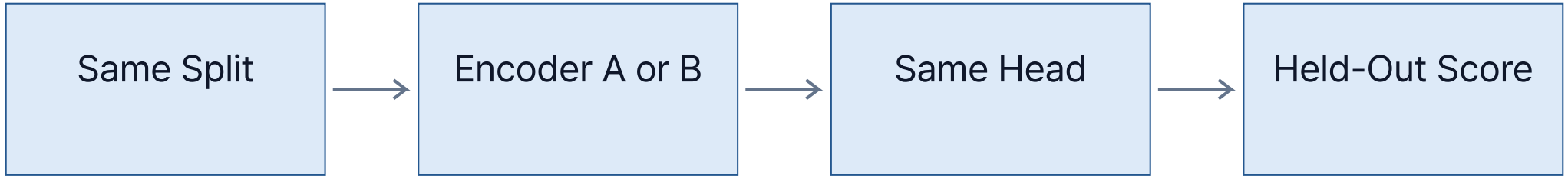
Run here | Stop | Reset

Recorded browser run · Run to recompute

Swap one image. Does every same-class pair have higher cosine?

Transfer Evaluation

Hold the data split and readout fixed across encoders



A controlled comparison changes one component

Representation Failure Audit

Check semantic, spatial and domain assumptions separately

Question	Check
Meaning	Inspect nearest examples
Location	Keep patch tokens
Domain	Evaluate shifted data

Takeaway: Visual Representations

01 Features determine what a small head can learn

02 Pair construction encodes an invariance assumption

03 Transfer requires a controlled held-out evaluation